

# 八岐大蛇

## sub1

暴力

## sub2

考虑将限制转化到树上，容易发现问题即为完全二叉树上拓扑序计数。

令  $dp_u$  表示以  $u$  为根的子树的拓扑序计数。容易发现点  $u$  一定在最前，而两个子树间的方案数互不影响。

因此得到递推式为  $dp_u = dp_{v1} * dp_{v2} * C(sz_{v1} + sz_{v2}, sz_{v1})$ ，其中  $v1, v2$  分别表示左右子树。

对于每次询问暴力dp求解即可，复杂度为  $O(nq)$ 。

## sub 3

整理上述式子可以得到方案数为  $\frac{n!}{\prod_{i=1}^n sz_i}$ ，其中  $sz_i$  表示点  $i$  的子树大小（其实这个式子对一般树同样适用）。

因此，我们只要在询问时维护所有节点  $sz_i$  的乘积就可以轻松求解。一个简单的做法是先预处理出每个节点的  $sz_i$ ，每次询问后暴力跳根修改父节点的  $sz_i$ ，由于完全二叉树层数为  $\log n$ ，时间复杂度为  $O(n + q \log^2 n)$ 。

## sub 4

考虑如何优化。 $n = 10^9$  的数据范围令我们难以直接求出每个子树的大小。但通过观察可以发现，完全二叉树存在大量相似的子树结构。具体来说，除了点  $n$  及其祖辈节点，其他节点的子树一定为满二叉树。

因此，我们预处理时完全可以直接统计不同层数的满二叉树子树个数，再集中计算。对于点  $n$  及其祖辈节点则直接特殊处理即可。同样由于完全二叉树层数为  $\log n$ ，因此预处理时间复杂度为  $O(\log^2 n)$ 。

另外上， $n = 10^9$  的阶乘难以直接计算（其实能算，但那就不是T1的难度了），因此还需要预处理出  $n = 10^9$  的阶乘。

## sub5

首先预处理直接使用上述方法即可。考虑修改，容易发现  $n = 10^9$  让我们难以在修改时直接维护  $sz_i$ 。但可以发现与询问有关的节点数量不会超过  $q \log n$  个。因此，我们可以提前离散化所有与询问有关的节点，并维护节点的父亲，子树大小。这样询问时就与上面的方法差不多了。

另外需要逆元的部分可以先求乘积，最后再逆元。而且需要注意特判相同的询问。时间复杂度为  $O(\log^2 n + q \log n)$  或  $O(q \log^2 n)$ 。

如果你不想对  $n = 10^9$  特判，而想获得一个  $n \leq 10^9$  的通用做法，可以使用分段打表计算  $n!$ 。

# 大豫通宝

## sub1

暴搜

## sub2

留给可能存在的建图判环/乱搞做法

## sub3

注意到，在黑格四联通的前提条件下，若干白格八连通等价于这些白格的“边界”八连通（其中边界是指周围八格内有黑格的白格）。而边界的数量是  $O(m)$  的，因此只需维护所有边界白格的连通性即可。

由于没有删除操作，每次添加黑格相当于删去一个白格，只需离线下来从后往前操作就变成添加，用并查集简单维护即可，复杂度  $O(m\alpha(m))$ 。

## sub4

以黑格为点，四相邻的黑格间连边转平面图，可以发现，每个白色八连通块会对应平面图中的一个面（即一个被点与边包围的封闭区域）。事实上，两者是“几乎”——对应的，唯一的例外是原图中黑色的“田”字（即共享同一个顶点的四个黑格）。

田字的数量可以简单维护，因此问题转化为对平面图中的面计数，套用平面图欧拉公式简单维护即可。使用哈希表，复杂度  $O(m \log m)$  或  $O(m)$ 。

请注意，在图中没有任何黑色格子的情况下，欧拉公式将不再适用，请特判并输出 0。

# 一丘之貉

## sub1

指数级暴力。

## sub2

状压暴力。

## sub3

复杂度不优秀的容斥。

## sub4

相当于求一个排列，任意两个相邻的数之差要  $> 1$ 。

钦定一些位置不满足要求并容斥。这些限制将整个序列分成了若干段，每段必须形如  $x, x+1, x+2, \dots$  或  $x, x-1, x-2, \dots$ ；设一共分了  $k$  段，有  $t$  段的长度  $> 1$ ，则方案数为  $2^t k!$ ，系数很简单就是  $(-1)^{n-k}$ 。

直接 dp 即可，复杂度  $O(n^2)$ 。

## sub5

复杂度不优秀的容斥。

## sub6

和 sub4 是几乎一样的，不过是搬到树上了而已。

考虑容斥，把一个方案  $p$  中的所有  $|p_u - p_v| = 1$  的边标红，所有红边形成一个链的剖分。我们要求不含任何红边的方案数。

钦定一些边是红色，其它边未知，可以算出方案。设最后红边形成  $k$  条链，且有  $t$  条链的长度  $> 1$ ，则方案数为  $2^t k!$ 。同时需要乘上  $(-1)^{n-k}$  作为容斥系数。

dp 可以在  $O(n^2)$  的复杂度内计算，具体的， $f_{x,k,0/1/2}$  表示  $x$  子树内有  $k$  条链，目前有  $0/1/2$  条与  $x$  相连的红边的方案数  $\times$  系数之和。

$2^t k!$  中  $2^t$  可以当成系数，在 dp 过程中直接记录，故不需要额外的复杂度来记录  $t$ ，这一步是平凡的，实在不会请参考一下给出的代码。

记得特判长度为 1 的链。

# 迷宫守卫

记  $d_i$  表示从  $i$  到根最短路上所有边的权值和;  $dis(x, y)$  表示从  $x$  到  $y$  最短路上所有边的权值和。

## sub1

首先应当注意到, 我们不会在 1 操作后进行 2 操作。所以我们的策略大概会是, 先随机到一个离  $t$  较近的点, 再直接走过去。

这启发我们不直接对某一个特定的  $s$  求答案, 因为在很多情况下随机一次之后就与  $s$  无关了。所以不妨从  $t$  入手。

对于一个特定的  $t$ , 不妨假设走到点集  $S$  内的点后, 接下来全部使用 1 操作。试着列出期望的表达式。

此时点集内的期望就是到  $t$  的距离, 点集外的点的期望可以表示为

$k \times \frac{n}{|S|} + \frac{\sum_{x \in S} dis(x, t)}{|S|}$ 。其中第一项是随机进点集的期望代价, 第二项是进点集之后移动的期望代价。

注意 1 操作只能往父亲走, 所以对于  $x \in S$ ,  $x$  一定是  $t$  的儿子, 即  $dis(x, t) = d_x - d_t$ , 不妨化简为  $\frac{nk + \sum_{x \in S} d_x}{|S|} - d_t$ 。其中  $nk$ ,  $d_t$  都可以视为常数。

子树内的点按照  $d_i$  排序后, 我们会选择一个前缀作为点集  $S$ , 显然只有  $O(n)$  种, 遍历一下就行。复杂度  $O(n^2 \log n)$  或  $O(n^2)$ 。

## sub2

留给常数过大的正解或复杂度为  $O(n \log^2 n)$  的做法。

## sub3

问题转化为, 给定一个递增序列  $a_{1 \sim m}$ , 和常数  $c = nk$ , 对于  $1 \leq r \leq m$ , 求

$f(r) = \frac{c + \sum_{i=1}^r a_i}{r}$  的最小值。

由于给定的树是一条链, 这个序列  $a$  可以直接由  $d$  的一个后缀  $d_{[n-m+1, n]}$  得到 (不必排序)。但  $m = O(n)$ , 依然不能直接遍历。

$f(r)$  是一个单谷函数, 可以直接三分。用前缀和优化一下计算过程, 时间复杂度  $O(n \log n)$ 。

$|S|$  是 dfn 序上一个区间, 可以简单将对每个  $t$  求改为对每个  $s$  求。

## sub4

考虑扩展到树上怎么做, 我们需要把子树内的所有  $d_i$  排序得到序列  $a$ 。

将按照 dfn 序拍成序列, 然后使用可持久化权值线段树维护区间并在其上二分即可。令  $f(x)$  表示取所有  $a_i \leq x$  的答案, 若  $f(x) < x + 1$ , 那么  $x + 1$  必然不存在于最终点集中, 反之则一定可以存在, 复杂度  $O(n \log n)$ 。

考虑如何将对每个  $t$  求转置为对每个  $s$  求。对于每个  $t$ ，计算出点集外的期望  $w$ ，相当于所有  $ans_i + = w$ ，然后对于  $x \in S$ ， $ans_x + = d_x - d_t - w$ 。

$x \in S$  可以表示为 dfn 序上，区间  $[l, r]$  内所有  $\leq k$  的位置，用扫描线维护即可，复杂度  $O(n \log n)$ 。

---

但是每次都重新求一遍集合看起来似乎有些多余，考虑每个节点的点集和自己儿子的点集有什么关系。

首先应当注意到，未被选入儿子点集中的点不会被选入当前节点的点集。证明考虑记儿子的答案为  $x$ ，当前这条边的边权是  $y$ ，我们直接继承儿子点集即可做到  $x + y$ 。而显然未加入的点距离均大于等于  $x + y$ ，这是不优的。

用可并堆维护集合，先合并所有儿子，然后不断试图移除最大值，时间复杂度同样是  $O(n \log n)$ 。效率应当是比主席树优秀许多的。